LLM 기반 Triton GPU 커널 자동 생성의 체계적 평가

Jemin Lee
Electronics and Telecommunications Research Institute
Daejeon, Republic of Korea
leejaymin@etri.re.kr

Abstract

GPU 커널 최적화는 고성능 컴퓨팅과 딥러닝 워크로드의 핵심 과제이며, OpenAI Triton은 Python 수준의 추상화로 GPU 프로그래밍의 접근성을 높였다. 본 연구는 대규모 언어 모델(LLM)을 활용한 Triton GPU 커널 자동 생성의 가능성과 한계를 체계적으로 평가한다. KernelBench 벤치마크의 250개 문제(Level 1–3)를 대상으로, 상용 모델 7종(gpt-4o-mini, gpt-4o, gpt-5.2, gpt-5.2-pro, gpt-5.4-nano, gpt-5.5, claude-opus-4-7)과 오픈소스 모델 1종(Qwen 2.5 Coder 32B)의 Triton 커널 생성 능력을 비교하였다. 에러 피드백 기반 반복적 개선(iterative refinement) 파이프라인을 설계하고, 다중 모델 캐스케이딩(cascading) 전략을 제안하여 5개 모델 합집합으로 51/100(51%)의 Level 1 정답률을 달성하였다. 실험 결과, claude-opus-4-7이 단일 모델 최고 성능(46/100, 46%)을 기록하였으며, gpt-5.5와 claude-opus-4-7이 처음 44문제에서 동일한 37개를 해결하는 플래그십 수렴 현상(Flagship Convergence)을 발견하였다. 에러 분석을 통해 컴파일 에러율이 모델 능력의 유효한 프록시(proxy)이고 값 불일치(value mismatch)가 정확도 향상의 핵심 병목임을 밝혔다. 반복 횟수 ablation 연구에서는 반복의 깊이보다 생성의 다양성이 더 효과적임을 보였다.

CCS Concepts: • **Software and its engineering → Just-in-time compilers**; **Software performance**; • **Computing methodologies → Machine learning approaches**.

Keywords: GPU kernel generation, Large language models, Triton, Code generation, KernelBench

ACM Reference Format:
Jemin Lee. 2026. LLM 기반 Triton GPU 커널 자동 생성의 체계적 평가. In . ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 서론

1.1 GPU 커널 최적화와 Triton의 등장

GPU 가속 컴퓨팅은 딥러닝, 과학 시뮬레이션, 대규모 데이터 처리 등 현대 컴퓨팅의 핵심 인프라로 자리잡았다. 특히 대규모 언어 모델(LLM)의 학습과 추론에서 GPU 커널의 성능은 전체 시스템 효율성을 좌우하는 결정적 요인이다 [7]. 그러나 고성능 GPU 커널을 작성하는 것은 메모리 계층 구조, 스레드 동기화, 워프(warp) 수준의 최적화 등 하드웨어에 대한 깊은 이해를 요구하는 전문적인 작업이다 [12]. 전통적으로 CUDA C/C++가 GPU 프로그래밍의 사실상 표준으로 사용되어 왔으나, 그 복잡성은 GPU 최적화의 보편적 접근을 어렵게 하는 주요 장벽이었다.

OpenAI Triton [26]은 이러한 문제를 해결하기 위해 제안된 Python 기반 GPU 프로그래밍 언어이다. Triton은 타일(tile) 기반 프로그래밍 모델을 통해 메모리 접근 패턴, 공유 메모리 관리, 스레드 스케줄링 등의 저수준 최적화를 컴파일러에 위임함으로써, 개발자가 알고리즘 로직에 집중할 수 있도록 한다. PyTorch 2.0 이후 `torch.compile`의 기본 백엔드로 채택되면서 Triton의 중요성은 더욱 증대되고 있다 [1]. 그럼에도 불구하고, 효율적인 Triton 커널을 작성하기 위해서는 타일 크기 선정, 메모리 coalescing 패턴, reduction 전략 등에 대한 상당한 전문 지식이 여전히 필요하다.

1.2 LLM 코드 생성 능력의 발전

대규모 언어 모델은 최근 수 년간 코드 생성 능력에서 괄목할 만한 발전을 이루었다. Codex [3]로부터 시작하여 CodeGen [18], StarCoder [16], GPT-4 [19]에 이르기까지, LLM은 자연어 명세로부터 다양한 프로그래밍 언어의 코드를 생성하는 능력을 지속적으로 향상시켜 왔다. HumanEval [3], MBPP [2], SWE-bench [10] 등의 벤치마크에서 최신 모델들은 전문 개발자에 필적하는 성능을 보이고 있다. 특히 GPT-5.2와 같은 최신 세대 모델은 복잡한 알고리즘 구현과 다중 파일에 걸친 코드 수정 능력에서 상당한 진전을 보여주고 있다.

그러나 이러한 LLM의 코드 생성 능력이 GPU 커널이라는 특수한 도메인에서 어느 정도 유효한지는 아직 충분히 규명되지 않았다. GPU 커널 코드는 일반적인 소프트웨어 코드와 달리 하드웨어 아키텍처에 대한 깊은 이해, 병렬 처리 패턴, 메모리 최적화 전략 등 도메인 특화 지식을 요구한다. 또한 생성된 커널의 평가는 단순한 기능적 정확성(correctness)을 넘어 성능(performance) 측면까지 고려해야 하므로, 일반적인 코드 생성 벤치마크와는 본질적으로 다른 평가 체계를 필요로 한다.

1.3 연구 문제

본 연구는 다음과 같은 핵심 연구 질문을 설정한다: *현재의 LLM은 PyTorch 레퍼런스 구현으로부터 기능적으로 정확한 (functionally correct) Triton GPU 커널을 얼마나 효과적으로 생성할 수 있는가?*

Conference'17, Washington, DC, USA
2026. ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

이 질문에 답하기 위해 우리는 KernelBench [21]를 평가 프레임워크로 채택하였다. KernelBench는 PyTorch로 작성된 GPU 연산의 레퍼런스 구현을 등가의 Triton 커널로 변환하는 과제를 정의하며, 난이도에 따라 Level 1(단일 연산, 100문제), Level 2(연산 조합, 100문제), Level 3(전체 아키텍처, 50문제)의 총 250개 문제를 포함한다. 각 문제는 레퍼런스 PyTorch 구현과 테스트 인프라를 제공하므로, 생성된 커널의 정확성을 자동으로 검증할 수 있다.

1.4 기존 연구의 한계

LLM 기반 GPU 커널 생성에 관한 기존 연구들은 몇 가지 한계를 지닌다. 첫째, KernelBench의 원 논문 [21]은 당시 사용 가능한 모델에 대한 기본적인 평가만을 제시하였으며, 최신 세대 모델(GPT-5.2 등)과 오픈소스 모델에 대한 비교가 부재하다. 둘째, 단일 모델의 성능 한계를 극복하기 위한 다중 모델 전략에 대한 체계적인 분석이 수행되지 않았다. 셋째, 반복적 개선 전략에서 반복 횟수와 성능 간의 관계, 즉 더 많은 반복이 더 나은 결과를 보장하는지에 대한 실증적 분석이 부족하다. 넷째, 추론(reasoning) 특화 모델이 코드 생성 과제에서 표준 모델 대비 어떤 특성을 보이는지에 대한 분석이 제한적이다. 마지막으로, 생성 실패의 원인을 체계적으로 분류하고 모델 간 실패 패턴의 차이를 분석한 연구가 부족하다.

1.5 연구 기여

본 연구의 주요 기여는 다음 네 가지로 요약된다.

1. **다중 LLM 모델의 체계적 비교 평가.** 상용 모델 7종(gpt-4o-mini, gpt-4o, gpt-5.2, gpt-5.2-pro, gpt-5.4-nano, gpt-5.5, claude-opus-4-7)과 오픈소스 모델 1종(Qwen 2.5 Coder 32B)에 대해 체계적 비교 평가를 수행하였다. claude-opus-4-7이 단일 모델 기준 최고 성능(46/100, 46%)을 달성하였으며, 추론 특화 모델(gpt-5.2-pro)은 코드 생성에 비효율적임을 확인하였다.
2. **다중 모델 캐스케이딩 전략 제안 및 검증.** 개별 모델의 정답 집합이 상이한 점에 착안하여, 다수의 모델을 순차적으로 적용하는 캐스케이딩 전략을 제안하였다. 5개 모델의 합집합으로 Level 1에서 51/100(51%)을 달성하였으며, 플래그십 수렴 현상(Flagship Convergence)을 발견하였다.
3. **반복 횟수 ablation 연구를 통한 생성 전략 분석.** 반복 횟수를 1, 3, 5, 10회로 변화시키며 성능 변화를 측정한 결과, 개별 실행의 최고 성능(22개)과 전체 합산 성능(28개) 사이에 유의미한 차이가 있음을 발견하였다. 이는 반복의 깊이(depth)보다 생성의 다양성(diversity)이 최종 성능 향상에 더 효과적임을 시사한다.
4. **에러 분류 체계 기반 실패 모드 분석.** 생성 실패를 컴파일 에러, 값 불일치(value mismatch), CUDA 에러 등으로 분류하는 체계를 수립하고, 모델별 에러 분포를 분석하였다. 컴파일 에러율이 모델의 Triton 코드 생성 능력을 나타내는 유효한 프록시(proxy)임을 밝혔으며, 값 불일치가 정확도 향상의 핵심 병목임을 확인하였다.

본 논문의 구성은 다음과 같다. 제 2절에서 관련 연구를 소개하고, 제 3절에서 시스템 설계를, 제 4절에서 실험 설정을 기술한다. 제 5절에서 실험 결과를 제시하고, 제 6절에서 논의를 수행한다. 마지막으로 제 7절에서 결론을 제시한다.

2 관련 연구

2.1 LLM 기반 코드 생성

대규모 언어 모델의 코드 생성 능력은 Codex [3]의 등장 이후 급속히 발전하였다. Codex는 HumanEval에서 28.8%의 pass@1을 달성하였으며, 이후 CodeGen [18], StarCoder [16] 등의 오픈소스 모델이 다양한 프로그래밍 언어에 대한 코드 생성 능력을 확장하였다. GPT-4 [19]는 HumanEval에서 67%의 pass@1을 달성하며 질적 도약을 이루었고, DeepSeek-Coder [8], CodeLlama [23], Qwen-Coder [9] 등의 오픈소스 모델도 상용 모델에 근접하는 성능을 보이고 있다. HumanEval [3], MBPP [2], SWE-bench [10] 등의 벤치마크가 코드 생성 능력 평가에 활용되고 있으나, 이들은 주로 범용 소프트웨어 개발에 초점을 맞추고 있어 GPU 커널과 같은 도메인 특화 코드 생성 평가에는 한계가 있다.

2.2 LLM 기반 DSL 코드 생성

LLM의 코드 생성 연구는 Python, Java 등 범용 언어에 집중되어 왔으나, 도메인 특화 언어(DSL)에 대한 코드 생성은 근본적으로 더 어려운 문제이다. DSL은 학습 데이터가 범용 언어 대비 극히 제한적이며, 도메인 고유의 의미론과 제약 조건에 대한 이해를 요구하기 때문이다. Thakur 등 [25]은 하드웨어 기술 언어인 Verilog를 대상으로 VeriGen 모델을 제안하여 LLM 기반 DSL 코드 생성의 가능성을 보였으며, Joel 등 [11]은 저자원 및 도메인 특화 언어에 대한 LLM 기반 코드 생성 연구를 체계적으로 정리하였다. GPU 프로그래밍 분야에서 Triton은 PyTorch 에코시스템과의 통합으로 빠르게 채택이 확산되고 있는 대표적인 DSL이며, FlashAttention [6, 7] 등 고성능 커널 구현에 핵심적으로 활용되고 있어 LLM 기반 자동 생성의 중요한 대상이 되고 있다.

2.3 GPU 커널 최적화 및 자동 생성

전통적인 GPU 커널 최적화는 컴파일러 및 탐색 기반 접근에 의존해 왔다. TVM [4]은 텐서 연산을 위한 컴파일러 프레임워크로 스케줄 기반의 최적화 탐색 공간을 정의하며, Ansor [30]는 자동 스케줄 탐색을 통해 수동 튜닝 없이 효율적 커널을 생성한다. Halide [22]는 알고리즘과 스케줄의 분리를 통해 이미지 처리 파이프라인의 최적화를 용이하게 한다.

한편, GPU 커널 개발의 생산성을 높이기 위한 도메인 특화 언어도 발전하고 있다. OpenAI Triton [26]은 블록 수준의 프로그래밍 모델을 채택하여 Python 수준의 사용성과 GPU 수준의 성능을 동시에 추구하며, ThunderKittens [24]는 하드웨어 추상화를 통해 간결하면서도 고성능인 AI 커널 작성을 지원한다. Mirage [28]는 다층 슈퍼옵티마이저로서 텐서 프로그램의 등가성 검증을 통한 자동 최적화를 수행한다. 이러한 접근들은 LLM을 사용하지 않고 컴파일러, 탐색, 또는 형식 검증 기법에 기반하며, 커널 코드 자체의 자동 생성보다는 기존 코드의 최적화에 초점을 맞추고 있다.

2.4 LLM 기반 GPU 커널 생성

LLM을 활용한 GPU 커널 생성 연구는 최근 급속히 확대되고 있다. 벤치마크 측면에서 KernelBench [21]는 PyTorch 레퍼런스로부터 등가의 최적화된 GPU 커널 생성을 평가하는 250개 문제의 3단계 벤치마크를 수립하였으며, TritonBench [15]는 184개의 실제 GitHub Triton 연산자를 수집하여 Triton에 특화된 벤치마크를 제공한다. Yu 등 [29]은 LLM 시대의 자동 커널 생성에 대한 포괄적인 서베이를 통해 이 분야의 연구 동향을 체계적으로 정리하였다.

강화 학습 기반 접근으로 TritonRL [27]은 LLM이 Triton 코드를 추론하고 생성하도록 RL 기반 훈련을 수행하며, AutoTriton [17]은 GRPO 기반 강화 학습을 통해 Triton 프로그래밍 능력을 향상시킨다. 프로파일링 기반 접근으로 TritonForge [14]는 실행 프로파일 정보를 활용하여 자동으로 Triton 커널을 최적화하는 프레임워크를 제안한다. CUDA 커널 생성 분야에서는 CUDA-LLM [5]이 LLM의 효율적 CUDA 커널 생성 능력을 입증하였으며, AI CUDA Engineer [13]는 에이전트 기반의 CUDA 커널 발견, 최적화, 합성 프레임워크를 제시한다.

본 연구는 이러한 기존 연구들과 여러 측면에서 차별화된다. 첫째, TritonRL이나 AutoTriton이 RL 기반의 모델 훈련을 필요로 하는 반면, 본 연구는 상용 API 모델을 추가 훈련 없이 그대로 활용하여 진입 장벽을 낮춘다. 둘째, 단일 모델 평가에 집중하는 기존 연구들과 달리 다중 모델 캐스케이딩 전략을 최초로 제안하고 그 효과를 실증한다. 셋째, 모델별 에러 분류 체계를 통한 실패 모드 분석과 확률적 다양성(stochastic diversity)이 반복 심화(iteration depth)보다 효과적이라는 발견을 제시한다.

2.5 추론 모델과 코드 생성

최근 o1 [20], gpt-5.2-pro 등의 추론(reasoning) 모델이 등장하여 수학적 추론과 복잡한 논리 문제에서 우수한 성능을 보이고 있다. 그러나 코드 생성 과제에서는 추가적인 추론 단계가 오히려 실행 가능한 코드 생성을 방해할 수 있다는 관찰이 보고되고 있다. 본 연구는 이러한 관찰을 GPU 커널 생성이라는 구체적 도메인에서 실증적으로 검증하며, gpt-5.2-pro가 Level 2 및 Level 3에서 0%의 정확도를 보이는 결과를 통해 추론 모델의 코드 생성 한계를 정량적으로 제시한다.

3 시스템 설계

본 절에서는 LLM 기반 Triton GPU 커널 자동 생성 시스템의 전체 구조와 각 구성 요소의 설계를 기술한다.

3.1 파이프라인 개요

제안하는 시스템은 PyTorch 레퍼런스 코드로부터 최적화된 Triton GPU 커널을 자동 생성하고 평가하는 엔드투엔드(end-to-end) 파이프라인으로 구성된다. Figure 1에 전체 파이프라인의 구조를 도시하였다. 파이프라인은 다음의 단계로 구성된다.

Figure 2에 제안 시스템의 전체 모듈 구조를 도시하였다.

1. **문제 로딩**: KernelBench [21]로부터 PyTorch 레퍼런스 코드를 포함한 문제를 로딩한다.
2. **프롬프트 구성**: `PromptBuilder`가 대상 GPU의 하드웨어 사양, Triton 프로그래밍 패턴, 시스템 프롬프트를 결합하여 LLM에 전달할 프롬프트를 구성한다.
3. **코드 생성**: LLM은 OpenAI API 또는 로컬 vLLM 서버를 통해 Triton 커널 코드를 생성한다.
4. **응답 파싱**: `ResponseParser`가 생성된 응답에서 코드 블록을 추출하고 클래스명 정규화 및 산문(prose) 제거 등의 후처리를 수행한다.
5. **평가**: `Evaluator`에 의해 정확성과 성능이 평가된다. 정확성 검증은 5개의 무작위 입력에 대해 레퍼런스 출력과의 절대 오차가 10^{-2} 이내인지 확인하며, 성능 벤치마크는 10회 시행의 중앙값으로 측정한다.
6. **반복적 개선**: 평가에 실패한 경우, `Reflector`가 에러를 분류하고 피드백을 생성하여 LLM에 재전달함으로써 반복적 개선(iterative refinement)을 수행한다. 이 과정은 최대 N회까지 반복된다.

3.2 프롬프트 설계

프롬프트 설계는 LLM이 올바른 Triton 커널을 생성하는 데 핵심적인 역할을 한다. 본 시스템의 프롬프트는 세 가지 구성 요소로 이루어진다.

시스템 프롬프트 GPU의 물리적 리소스 제약 조건을 명시적으로 포함한다. 특히, 공유 메모리의 크기가 약 100KB(101,376 bytes)로 제한되며, `BLOCK_SIZE`를 적절히 설정해야 한다는 가이드라인을 포함한다. 예를 들어, 행렬 곱셈의 경우 `BLOCK_SIZE=64` 이하, element-wise 연산의 경우 `BLOCK_SIZE=1024`를 권장한다.

사용자 프롬프트 PyTorch 레퍼런스 코드 전체와 대상 GPU의 사양(이름, 아키텍처, VRAM 용량, 메모리 대역폭, 이론적 TFLOPS)을 포함한다.

개선 프롬프트 반복적 개선 단계에서 사용되며, 이전에 생성된 코드, 발생한 에러 메시지, 현재 반복 횟수를 포함한다. 이를 통해 LLM은 이전 시도의 실패 원인을 인지하고 수정된 코드를 생성할 수 있다.

출력 형식은 단일 Python 코드 블록으로 제한하며, `torch.nn.Module`을 상속하는 `Model` 클래스와 `@triton.jit` 데코레이터가 적용된 커널 함수를 포함하도록 요구한다.

3.3 CUDA 에러 격리

Triton 커널 평가 과정에서 발견된 중요한 기술적 문제는 CUDA 에러의 전파(cascading) 현상이다. 하나의 잘못된 커널이 실행 중 CUDA 컨텍스트를 오염시키면, 동일 프로세스 내에서 실행되는 모든 후속 커널 평가가 실패하게 된다. 이는 CUDA 런타임이 프로세스 수준에서 상태를 관리하기 때문에 발생하는 근본적인 제약이다.

이 문제를 해결하기 위해, 각 커널 평가를 독립적인 서브프로세스(subprocess)에서 실행하는 격리 메커니즘을 설계하였다. 서브프로세스는 커널을 로딩하고 정확성 및 성능 평가를 수행한 후, 결과를 JSON 형식으로 표준 출력(stdout)

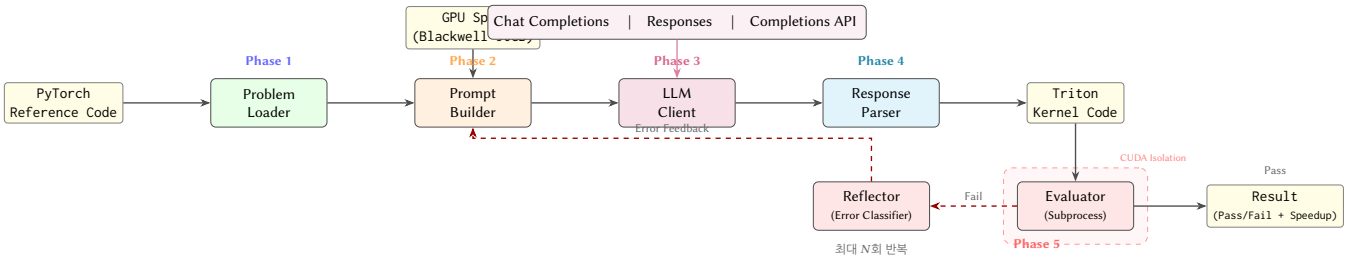


Figure 1. LLM 기반 Triton GPU 커널 자동 생성 시스템의 전체 파이프라인. PyTorch 레퍼런스 코드로부터 프롬프트를 구성하고, LLM이 Triton 커널을 생성하며, 평가 실패 시 에러 피드백을 통해 반복적으로 개선한다. 각 커널 평가는 CUDA 에러 격리를 위해 독립적인 서브프로세스에서 수행된다.

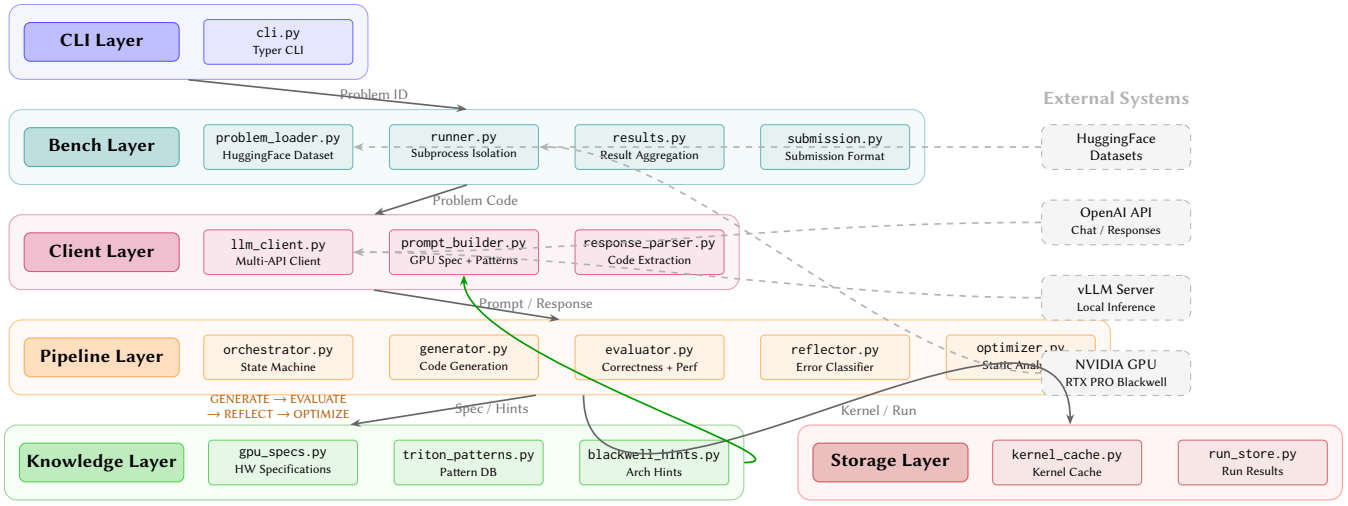


Figure 2. 제안 시스템의 전체 모듈 구조. CLI 레이어가 사용자 입력을 받아 Bench 레이어에서 KernelBench 문제를 로딩하고, Client 레이어가 프롬프트 구성 및 LLM 응답 파싱을 담당한다. Pipeline 레이어의 상태 머신(GENERATE → EVALUATE → REFLECT → OPTIMIZE)이 반복적 생성-평가 루프를 조율하며, Knowledge 레이어의 GPU 사양 및 Triton 패턴이 프롬프트 품질을 향상시킨다. 생성된 커널과 실험 결과는 Storage 레이어에 저장된다.

에 기록한다. 부모 프로세스는 서브프로세스의 출력을 파싱하여 평가 결과를 수집한다. 이 설계는 250개 문제를 순차적으로 평가하는 대규모 실험에서 필수적이며, 이를 도입하지 않을 경우 단일 실패가 전체 실험 결과를 무효화할 수 있다.

3.4 다중 모델 캐스케이딩 전략

단일 모델의 한계를 극복하기 위해, 복수의 LLM을 순차적으로 활용하는 캐스케이딩(cascading) 전략을 설계하였다.

- **Phase 1**: 전체 문제 집합에 대해 gpt-4o를 실행한다.
- **Phase 2**: Phase 1에서 실패한 문제에 대해 gpt-5.2를 실행한다.
- **Phase 3**: Phase 1과 2 모두에서 실패한 문제에 대해 gpt-5.2-pro를 실행한다.

최종 결과는 각 단계에서 해결된 문제 집합의 합집합(union)으로 산출된다. 이 접근법은 각 모델이 서로 다른 유형의 문제에 강점을 보인다는 관찰에 기반한다. Figure 3에 캐스케이딩 전략의 구조와 각 단계별 성능을 도시하였다.

3.5 다중 API 지원

본 시스템은 다양한 LLM 제공자의 API를 통합적으로 지원하기 위해 자동 API 감지 및 폴백(fallback) 메커니즘을 구현하였다.

- **Chat Completions API**: gpt-4o-mini, gpt-4o, gpt-5.2 등 일반적인 채팅 모델에 사용된다.
- **Responses API**: gpt-5.2-pro와 같은 추론 모델에 사용된다. `temperature` 파라미터를 지원하지 않으며, 시스템은 모델 식별자를 기반으로 이를 자동으로 감지한다.

- **Completions API**: 로컬 vLLM 서버에서 제공하는 오픈소스 모델(예: Qwen 2.5 Coder)에 사용된다.

이러한 다중 API 지원은 상용 모델과 오픈소스 모델을 단일 프레임워크 내에서 일관되게 비교 평가할 수 있게 한다.

3.6 구현 기여

본 연구의 시스템은 약 2,500 라인의 Python 코드로 구현되었으며, 재현 가능한 오픈소스 프레임워크로 공개하였다. 구현상의 주요 기술적 기여는 다음과 같다.

***GPU* 인식 프롬프트 엔지니어링** 프롬프트에 대상 GPU의 공유 메모리 크기(101,376 bytes), VRAM 용량, 메모리 대역폭, 이론적 TFLOPS 등 하드웨어 사양을 명시적으로 포함한다. 또한 Triton 프로그래밍 패턴 데이터베이스(`triton_patterns.py`)와 Blackwell 아키텍처 최적화 힌트(`blackwell_hints.py`)를 프롬프트에 주입하여, LLM이 하드웨어 특성에 맞는 커널을 생성하도록 유도한다.

자동 *API* 감지 및 폴백 `LLMClient`는 모델 식별자를 기반으로 Chat Completions API, Responses API, Completions API 중 적절한 API를 자동으로 선택한다. 추론 모델(gpt-5.2-pro, o1, o3)은 `temperature` 파라미터를 지원하지 않으므로, 이를 자동으로 감지하여 Responses API로 전환한다. 비채팅 모델은 Completions API로 폴백하며, 한번 감지된 모델 유형은 캐싱하여 이후 호출에서 재시도를 방지한다.

응답 후처리 파이프라인 `ResponseParser`는 LLM 응답에서 Markdown 코드 블록을 추출하고, 클래스명을 `Model`로 정규화하며, Triton 코드 외의 산문(prose)을 제거한다. 이 후처리는 다양한 LLM의 출력 형식 차이를 흡수하여 일관된 평가를 가능하게 한다.

상태 머신 기반 오케스트레이션 `Orchestrator`는 GENERATE → EVALUATE → REFLECT → OPTIMIZE의 4단계 상태 머신으로 구현되며, 각 단계의 전이 조건과 최대 반복 횟수를 설정 가능하게 설계하였다. `Reflector`는 에러 메시지를 분석하여 컴파일 에러, 값 불일치, CUDA 에러, 타임아웃 등으로 자동 분류하고, 에러 유형에 맞는 피드백 프롬프트를 생성한다.

4 실험 설정

4.1 벤치마크: KernelBench

본 연구의 평가에는 KernelBench [21]를 사용한다. KernelBench는 PyTorch로 작성된 레퍼런스 코드를 등가의 Triton GPU 커널로 변환하는 능력을 측정하기 위해 설계된 벤치마크로, HuggingFace datasets를 통해 제공된다.

- **Level 1** (100 문제): 단일 연산 수준의 문제로, element-wise 연산, reduction, 행렬 곱셈 등 기본적인 GPU 연산을 포함한다.
- **Level 2** (100 문제): 복수 연산의 조합으로 구성된 문제로, attention 메커니즘, convolution 연산 등 중간 수준의 복잡도를 갖는다.
- **Level 3** (50 문제): 전체 모델 또는 모델 블록 수준의 문제로, ResNet 블록, Transformer 레이어 등을 포함하며 가장 높은 난이도를 갖는다.

Table 1. 실험에 사용된 LLM 모델.

모델	유형	API	파라미터	반복 횟수
gpt-4o-mini	Chat	Chat Completions	비공개	5
gpt-4o	Chat	Chat Completions	비공개	5
gpt-5.2	Chat	Chat Completions	비공개	5
gpt-5.2-pro	Reasoning	Responses	비공개	3
Qwen 2.5 Coder	Chat	Completions (vLLM)	32B	5
gpt-5.4-nano	Chat	Chat Completions	비공개	3
gpt-5.5	Chat	Chat Completions	비공개	3
claude-opus-4-7	Chat	Chat Completions	비공개	3

4.2 평가 모델

Table 1은 본 실험에서 평가한 LLM의 목록이다.

OpenAI의 상용 모델은 파라미터 수가 공개되지 않으며, API를 통해서만 접근 가능하다. Qwen 2.5 Coder 32B [9]는 로컬 vLLM 서버에서 실행하였다. gpt-5.2-pro는 추론(reasoning) 특화 모델로, API 호출 비용과 응답 지연을 고려하여 반복 횟수를 3회로 설정하였다. 확장 실험에서 추가된 gpt-5.4-nano, gpt-5.5, claude-opus-4-7은 Mindlogic AI Gateway[1]를 통합 API 엔드포인트로 사용하여 평가하였으며, 반복 횟수는 3회로 설정하였다.

4.3 평가 메트릭

KernelBench에서 정의한 세 가지 메트릭을 사용한다 [21].

- **Fast-0 (정답률)**: 생성된 커널이 정확한 결과를 출력하는 비율. 5개의 무작위 입력에 대해 레퍼런스 출력과의 절대 오차(absolute tolerance)가 10^{-2} 이내이면 정답으로 판정한다.
- **Fast-1**: 정답인 동시에 speedup ≥ 1.0×인 비율.
- **Fast-2**: 정답인 동시에 speedup ≥ 2.0×인 비율.

Speedup은 speedup = $t_{\text{ref}}/t_{\text{gen}}$으로 계산하며, 각 시간은 10회 시행의 중앙값으로 측정한다.

4.4 하드웨어 환경

모든 실험은 동일한 하드웨어 환경에서 수행되었다. GPU는 NVIDIA RTX PRO 6000 Blackwell로, 96GB VRAM과 sm_120 컴퓨트 아키텍처를 탑재하고 있다. 소프트웨어 환경은 Triton 3.5.1, PyTorch 2.x, Python 3.12, CUDA 12.x로 구성하였다.

4.5 실험 설계

실험 *1*: 모델 간 비교 5개 모델에 대해 KernelBench의 3개 레벨 전체(250개 문제)를 평가한다. 다중 모델 캐스케이딩 전략의 효과를 측정하기 위해 gpt-4o, gpt-5.2, gpt-5.2-pro를 순차적으로 적용한 결과를 분석한다. 확장 실험으로 gpt-5.4-nano, gpt-5.5, claude-opus-4-7을 Level 1(100문제)에 대해 추가 평가하고, 5개 모델 합집합의 성능을 분석한다.

실험 *2*: 반복 횟수 *ablation*. gpt-4o 모델과 Level 1 문제를 대상으로 반복 횟수를 $\{1, 3, 5, 10\}$으로 변화시키며 실험한다.

[1] https://api.mindlogic.ai

Table 2. 모델별 KernelBench 정확도 비교(Fast-0). †gpt-4o 실패 문제에 대해서만 실행(cascading strategy). ‡Level 1만 평가(확장 실험).

Model	L1 (100)	L2 (100)	L3 (50)	Total (250)
Qwen 2.5 Coder 32B	0 (0%)*	—	—	0*
gpt-4o-mini	15 (15%)	1 (1%)	0 (0%)	16 (6.4%)
gpt-4o	19 (19%)	6 (6%)	0 (0%)	25 (10%)
gpt-5.4-nano‡	35 (35%)	—	—	—
gpt-5.2†	23/81 (28.4%)	9/94 (9.6%)	4/50 (8%)	36/225 (16%)
gpt-5.5‡	37 (37%)	—	—	—
gpt-5.2-pro†	13/81 (16%)	0/94 (0%)	0/50 (0%)	13/225 (5.8%)
claude-opus-4-7‡	**46 (46%)**	—	—	—

*파이프라인 초기 버전의 응답 파서 결함으로 모든 출력이 구문 오류로 실패. 모델 자체의 코드 생성 능력을 반영하지 않음.

실험 3: 에러 분석 전체 실패 사례의 에러 유형을 분류하고 레벨별, 모델별 에러 분포를 분석한다.

5 실험 결과

5.1 모델 비교 결과

Table 2은 각 모델의 난이도별 정확도를 보여준다.

확장 실험을 포함한 전체 결과에서, **claude-opus-4-7이 단일 모델 최고 성능(46/100, 46%)**을 달성하였다. 이는 기존 최고였던 gpt-5.2의 캐스케이딩 기준 28.4%를 크게 상회하며, 이전 3-모델 캐스케이딩 전략(43/100)조차 초과하는 결과이다. gpt-5.5(37/100, 37%)와 gpt-5.4-nano(35/100, 35%)도 기존 gpt-4o(19%)를 크게 상회하여, 최신 세대 모델의 Triton 커널 생성 능력이 전반적으로 향상되었음을 확인하였다.

특히 주목할 현상은 gpt-5.5와 claude-opus-4-7이 처음 44문제(P1–P44)에서 **정확히 동일한 37개 문제**를 해결했다는 점이다. P4만을 유일한 공통 실패로 공유하며 나머지 43문제에서 완벽히 일치하는 성공/실패 패턴을 보였다. 그러나 P45 이후 후반부에서 두 모델은 극적으로 분기하였다: gpt-5.5는 추가 0개, claude-opus-4-7은 추가 9개(P88–P100)를 해결하였다. 이러한 **후반부 회복 능력(late recovery)**이 claude-opus-4-7을 단일 모델 최고 성능으로 이끈 핵심 차별화 요인이다.

기존 모델 중 gpt-5.2는 시도한 문제 대비 높은 성공률을 달성하였으며, Level 3에서도 유일하게 4문제를 해결하여 복잡한 커널에 대한 생성 능력이 우수함을 보여준다. 반면, gpt-5.2-pro는 추론 특화 모델(reasoning model)임에도 불구하고 코드 생성 태스크에서 기대 이하의 성능을 보였다. Level 1에서 16%에 그쳤으며, Level 2와 Level 3에서는 단 하나의 문제도 해결하지 못하였다. 이는 해당 모델의 긴 추론 시간으로 인한 타임아웃이 주요 원인이다(Section 5.4 참조).

Level 3은 gpt-4o와 gpt-4o-mini 모두 50문제 중 단 하나도 해결하지 못하여, 심각한 난이도 절벽(difficulty cliff)이 존재함을 확인하였다. Qwen 2.5 Coder 32B의 경우, 파이프라인 초기 버전의 응답 파서 결함으로 인해 모든 생성 결과가 구문 오류로 실패하여 공정한 비교가 불가능하였다.

Table 3. 다중 모델 캐스케이딩 전략의 누적 성능.

Configuration	L1 (/100)	L2 (/100)	L3 (/50)	Total (/250)
gpt-4o-mini	15 (15%)	1 (1%)	0 (0%)	16 (6.4%)
gpt-4o	19 (19%)	6 (6%)	0 (0%)	25 (10%)
gpt-4o + gpt-5.2	42 (42%)	15 (15%)	4 (8%)	61 (24.4%)
+ gpt-5.2-pro	43 (43%)	15 (15%)	4 (8%)	62 (24.8%)
claude-opus-4-7 (단일)	46 (46%)	—	—	—
opus + gpt-5.4-nano	48 (48%)	—	—	—
전체 5모델 union	**51 (51%)**	—	—	—

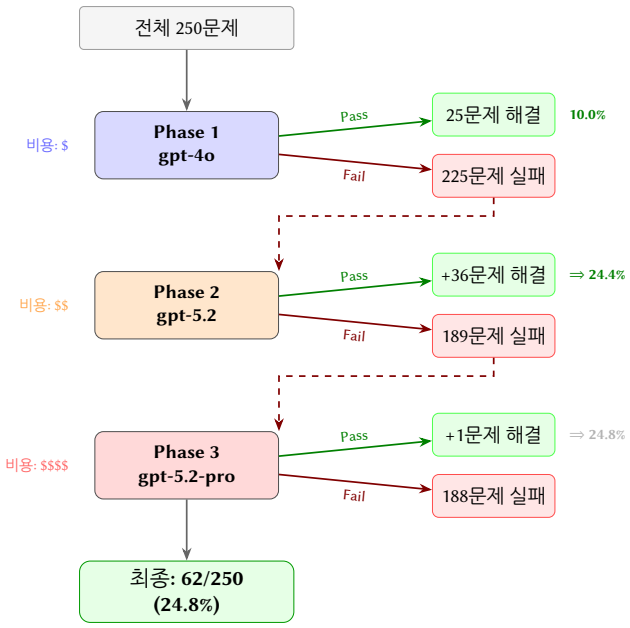


Figure 3. 다중 모델 캐스케이딩 전략. 각 단계에서 이전 단계의 실패 문제만을 대상으로 상위 모델을 적용한다. gpt-4o→gpt-5.2 단계에서 2.44배의 성능 향상이 달성되며, gpt-5.2-pro 추가의 비용 대비 효과는 미미하다.

5.2 다중 모델 캐스케이딩 결과

서로 다른 LLM이 질적으로 다른 문제를 해결할 수 있다는 가설 하에, 다중 모델 캐스케이딩 전략을 실험하였다. Table 3에 결과를 정리하였다.

기존 캐스케이딩 전략의 효과는 현저하다. gpt-4o 단독 25문제에서 gpt-5.2를 추가함으로써 61문제로 증가하여, 36개의 추가 문제를 해결하였다(**2.44배 향상**). 이는 gpt-5.2가 gpt-4o와 질적으로 다른 문제 해결 능력을 보유함을 의미한다. 구체적으로, Level 1에서 23개, Level 2에서 9개, Level 3에서 4개의 추가 문제를 해결하였다.

반면, gpt-5.2-pro의 추가 기여는 미미하였다. 전체 225문제를 시도하여 단 1개의 추가 문제(Level 1의 Problem 88, 7.60× speedup)만을 해결하였다. 이는 높은 API 비용과 긴 응답 시간을 고려할 때, 캐스케이딩 전략에서의 비용 대비 효과가 극히 낮음을 시사한다.

확장 실험에서는 최신 모델을 포함한 합집합 분석을 수행하였다. claude-opus-4-7 단일 모델(46/100)이 이전 3-모델 캐스케이딩(43/100)을 초과한 것이 가장 주목할 결과이다.

Table 4. 반복 횟수에 따른 성능 및 에러 유형 변화 (gpt-4o, Level 1).

Iter.	Solved (/100)	Compile Err.%	Val. Mismatch%
1	19 (19%)	59.3%	24.7%
3	21 (21%)	51.9%	39.2%
5	19 (19%)	45.7%	37.0%
10	22 (22%)	32.1%	53.8%
Union	**28 (28%)**	—	—

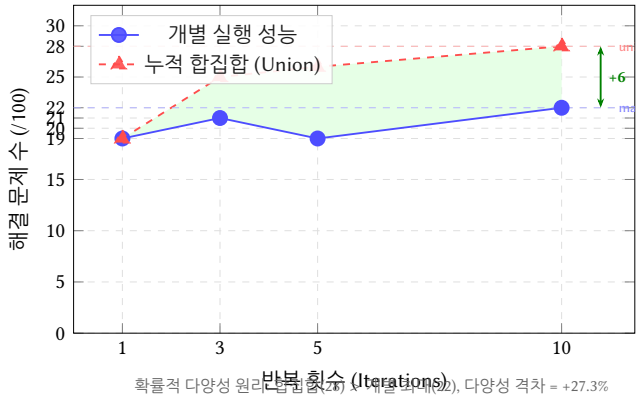


Figure 4. 반복 횟수에 따른 성능 변화 (gpt-4o, Level 1). 개별 실행 성능은 반복 횟수에 따라 미미한 변동을 보이나, 누적 합집합은 꾸준히 증가한다. 이는 반복의 깊이보다 생성의 다양성이 더 효과적임을 시사한다.

claude-opus-4-7과 gpt-5.4-nano의 2-모델 합집합은 48/100으로, gpt-5.4-nano가 opus가 해결하지 못한 2개의 추가 문제를 해결하였다. 5개 모델 전체의 합집합은 **51/100(51%)**으로, 최선의 단일 모델(46/100) 대비 10.9% 향상을 달성하여, 확률적 다양성 원리가 최신 세대 모델에서도 유효함을 확인하였다.

5.3 반복 횟수 Ablation 연구

반복적 개선 전략에서 반복 횟수가 성능에 미치는 영향을 분석하기 위해, gpt-4o 모델로 Level 1 문제에 대한 ablation 실험을 수행하였다. Table 4에 결과를 제시한다.

주목할 점은 개별 실행의 최대 성공 수(22개, 10회 반복)와 합집합(28개) 사이의 격차이다. 이는 각 실행이 서로 다른 문제를 확률적으로 해결함을 의미하며, 반복 깊이를 늘리는 것보다 독립적인 다수 시도를 통한 **확률적 다양성(stochastic diversity)**이 더 효과적인 전략임을 시사한다.

에러 유형의 변화 추이도 의미있다. 반복 횟수가 증가함에 따라 컴파일 에러 비율은 59.3%에서 32.1%로 꾸준히 감소하는 반면, 값 불일치 비율은 24.7%에서 53.8%로 증가한다. 이는 반복적 개선이 Triton 문법 오류를 효과적으로 수정하지만, 의미적 정확성(semantic correctness) 문제는 반복만으로 해결되지 않음을 보여준다.

Table 5. 모델별 주요 실패 유형 요약.

Model	Primary Failure	Secondary	해석
gpt-4o-mini	Compile (71–82%)	Val. mismatch	Triton 문법 미숙
gpt-4o	Compile (38–51%)	Val. mismatch	문법 개선, 의미 정확성 부족
gpt-5.2	Val. mismatch (59–72%)	Compile (7–26%)	문법 능숙, 의미적 병목
gpt-5.2-pro	Timeout (82–100%)	Rate limit	추론 오버헤드
gpt-5.4-nano	Val. mismatch + Other	CUDA	혼합 에러 프로필
gpt-5.5	Compile (71%)	Timeout (25%)	Triton 문법 미숙
claude-opus-4-7	Val. mismatch (65%)	Timeout (28%)	문법 완벽, 의미적 병목

Table 6. 모델 및 난이도별 에러 분포(%). 실패한 문제 중 해당 에러 유형의 비율.

Model	Level	Failed	Compile	Val. Mis.	CUDA	Timeout
gpt-4o-mini	L1	85	71.8%	20.0%	2.4%	2.4%
	L2	99	70.7%	22.2%	4.0%	1.0%
	L3	50	82.0%	6.0%	6.0%	0%
gpt-4o	L1	81	45.7%	37.0%	11.1%	0%
	L2	94	51.1%	41.5%	4.3%	0%
	L3	50	38.0%	56.0%	4.0%	0%
gpt-5.2	L1	58	6.9%	58.6%	1.7%	20.7%
	L2	85	25.9%	62.4%	0%	8.2%
	L3	46	17.4%	71.7%	0%	4.3%
gpt-5.2-pro	L1	68	0%	0%	0%	100%
	L2	94	0%	0%	0%	81.9%
	L3	50	0%	0%	0%	0%*
gpt-5.4-nano	L1	65	—	—	—	—
gpt-5.5	L1	63	71%	3%	0%	25%
claude-opus-4-7	L1	54	0%	65%	0%	28%

*Level 3의 gpt-5.2-pro 실패는 전량 API rate limit(HTTP 429) 에러에 기인.

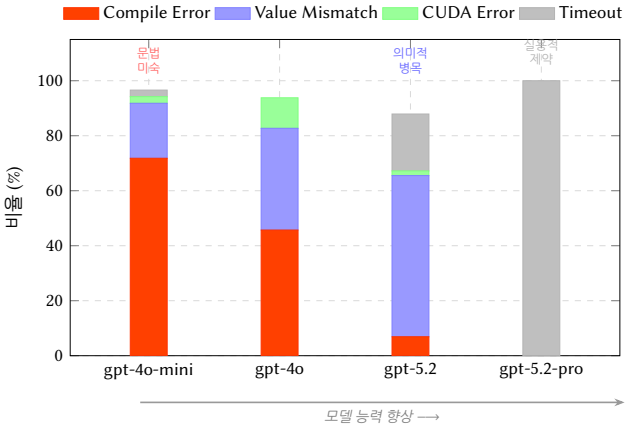


Figure 5. 모델별 에러 유형 분포 (Level 1 기준). 모델 능력이 향상됨에 따라 컴파일 에러에서 값 불일치(value mismatch)로 지배적 에러 유형이 전이되는 패턴이 관찰된다. gpt-5.2-pro의 실패는 전량 타임아웃에 기인한다.

5.4 에러 분석

생성 실패의 원인을 체계적으로 분류하기 위해, 각 모델의 에러 유형 분포를 분석하였다. Table 5에 요약을, Table 6에 상세 분포를 제시한다.

에러 분석에서 도출되는 핵심 통찰은 다음과 같다.

Table 7. 상위 10개 속도 개선 결과.

Rank	Problem	Level	Model	Speedup
1	P16	3	gpt-5.2	84.03×
2	P9	2	gpt-4o-mini	25.45×
3	P70	2	gpt-4o	19.10×
4	P12	1	gpt-4o	11.86×
5	P88	1	gpt-5.2-pro	7.60×
6	P100	1	gpt-5.2-pro	7.35×
7	P49	2	gpt-4o	6.54×
8	P98	1	gpt-5.2-pro	5.32×
9	P43	3	gpt-5.2	4.26×
10	P99	1	gpt-5.2	4.05×

Table 8. 모델별 Fast-1 (speedup ≥ 1.0×) 및 Fast-2 (speedup ≥ 2.0×) 지표.

Model	Fast-1	Fast-2
gpt-4o-mini	7/16 (43.8%)	1/16 (6.3%)
gpt-4o	12/25 (48.0%)	4/25 (16.0%)
gpt-5.2	25/36 (69.4%)	5/36 (13.9%)
gpt-5.2-pro	10/13 (76.9%)	3/13 (23.1%)
gpt-5.4-nano	21/35 (60.0%)	10/35 (28.6%)
gpt-5.5	22/37 (59.5%)	8/37 (21.6%)
claude-opus-4-7	29/46 (63.0%)	14/46 (30.4%)

첫째, **컴파일 에러율은 모델 능력과 역비례한다**: gpt-4o-mini(71–82%) > gpt-4o(38–51%) > gpt-5.2(7–26%). 특히 gpt-5.2의 Level 1 컴파일 에러율이 6.9%에 불과한 것은 Triton 문법을 거의 완벽하게 습득하였음을 의미한다.

둘째, 컴파일 에러가 감소하면 **값 불일치가 지배적 에러로 부상한다**. gpt-5.2에서 값 불일치가 59–72%를 차지하는 것은, 현재 LLM 기반 커널 생성의 핵심 도전이 Triton 문법이 아닌 GPU 연산의 논리적 이해에 있음을 시사한다.

셋째, gpt-5.2-pro의 실패는 모델 능력이 아닌 **실용적 제약에 기인한다**. 추론 모델의 긴 사고 과정(chain-of-thought)이 타임아웃을 유발하여, Level 2에서 81.9%, Level 1에서 100%의 문제가 타임아웃으로 실패하였다.

5.5 속도 개선(Speedup) 분석

정확성 평가를 통과한 커널에 대해 PyTorch 기준 구현 대비 실행 속도 개선을 측정하였다. Table 7에 상위 10개 결과를, Table 8에 Fast-1 및 Fast-2 지표를 제시한다.

가장 주목할 만한 결과는 Level 3 Problem 16에서 gpt-5.2가 달성한 84.03배의 속도 개선이다. 이는 특정 연산 패턴에서 LLM이 생성한 Triton 커널이 PyTorch의 기본 구현을 극적으로 능가할 수 있음을 보여준다.

Fast-1 지표에서 gpt-5.2(69.4%)와 gpt-5.2-pro(76.9%)가 gpt-4o-mini(43.8%)와 gpt-4o(48.0%)를 크게 상회한다. 흥미로운 점은 gpt-5.2-pro가 해결한 문제 수는 13개로 적지만, Fast-2 비율(23.1%)은 가장 높다는 것이다. 이는 추론 모델이 문제를 해결할 경우 보다 최적화된 커널을 생성하는 경향이 있음을 시사하나, 낮은 전체 성공률을 고려하면 실용적 가치는 제한적이다.

6 논의

6.1 핵심 발견: 에러 전이 패턴

본 연구의 가장 중요한 발견은 LLM의 Triton 코드 생성 능력이 향상됨에 따라 에러 유형이 체계적으로 전이된다는 것이다. 이를 **에러 전이 가설(Error Transition Hypothesis)**로 정리한다: 모델 능력이 향상될수록 컴파일 에러(문법적 실패)에서 값 불일치(의미적 실패)로 지배적 에러 유형이 전이된다(Figure 5). gpt-4o-mini의 컴파일 에러율 71–82%에서 gpt-5.2의 7–26%로의 감소와, 동시에 값 불일치의 20%에서 59–72%로의 증가는 이 가설을 강하게 뒷받침한다. 이는 LLM이 Triton DSL의 문법을 학습하는 것은 비교적 용이하나, GPU 연산의 수치적 의미론(numerical semantics)을 이해하는 것은 본질적으로 어려운 과제임을 시사한다.

이 발견은 향후 LLM 기반 커널 생성 연구의 방향을 제시한다. 현재 대부분의 반복적 개선 전략은 에러 메시지를 피드백으로 제공하는 방식이나, 값 불일치 에러의 경우 에러 메시지만으로는 수정 방향을 결정하기 어렵다. 따라서 중간 결과 비교, 텐서 형상(shape) 추적, 수치 안정성 분석 등 보다 풍부한 피드백 메커니즘이 필요하다.

6.2 핵심 발견: 확률적 다양성의 우위

반복 횟수 ablation 연구에서 발견한 **확률적 다양성 원리(Stochastic Diversity Principle)**는 실용적으로 중요한 시사점을 제공한다. 4개 독립 실행의 합집합(28개)이 최선의 단일 실행(22개)을 27.3% 상회하며, 각 실행이 고유하게 해결하는 문제가 존재한다(Table 4). 이는 LLM의 코드 생성이 본질적으로 확률적(stochastic) 과정이며, 동일 문제에 대해서도 다른 경로의 해를 탐색할 수 있음을 의미한다.

이 원리의 실용적 귀결은 명확하다: 단일 모델로 깊이(depth) 있는 반복보다, 여러 독립적 시도를 통한 넓이(breadth) 우선 탐색이 더 비용 효율적이다. 이는 best-of-N 전략이나 다중 모델 캐스케이딩 전략의 이론적 근거를 제공하며, 실제로 gpt-4o+gpt-5.2 캐스케이딩이 2.44배 향상을 달성한 결과와 일맥상통한다.

6.3 비용 효율성 분석

본 실험 결과는 고비용 추론 모델이 반드시 우수한 성능을 보장하지 않음을 명확히 보여준다. gpt-5.2-pro는 gpt-5.2 대비 수 배 높은 API 비용과 긴 응답 시간에도 불구하고, 캐스케이딩 전략에서 단 1개의 추가 문제만 해결하였다. 코드 생성 태스크에서는 추론 깊이보다 코드 패턴에 대한 학습 범위가 더 중요한 요인이다.

실험 결과를 종합하면, gpt-4o와 gpt-5.2의 2단계 캐스케이딩이 비용 대비 최적의 전략이다. gpt-5.2-pro의 추가는 0.4%p의 미미한 향상만을 제공하므로, 비용 효율 관점에서 권장되지 않는다.

6.4 핵심 발견: 플래그십 수렴 현상

확장 실험에서 발견한 가장 흥미로운 현상은 gpt-5.5와 claude-opus-4-7이 처음 44문제에서 **정확히 동일한 37개 문제**를 해결했다는 점이다. 두 모델은 서로 다른 회사(OpenAI vs Anthropic)에서 독립적으로 개발되었음에도, P4만을 유일한 공통 실패로 공유하며 나머지 43문제에서 완벽하게 일치하는 성공/실패 패턴을 보였다.

이러한 **플래그십 수렴 현상(Flagship Convergence)**은 최상위 모델들이 일정 난이도 이하의 Triton 커널 생성 문제에 대해 사실상 동일한 능력 수준에 도달했음을 시사한다. 그러나 P45 이후 후반부에서 두 모델은 극적으로 분기하였다: gpt-5.5는 추가 0개, claude-opus-4-7은 추가 9개(P88–P100)를 해결하였다. 이 **후반부 회복 능력(late recovery)**이 claude-opus-4-7을 단일 모델 최고 성능(46/100)으로 이끈 핵심 차별화 요인이다.

또한, claude-opus-4-7 단일 모델(46/100)이 이전 3-모델 캐스케이딩 전략(43/100)을 초과한 것은 주목할 만하다. 이는 최신 세대 플래그십 모델의 발전이 다중 모델 전략의 필요성을 줄일 수 있음을 시사한다. 그러나 5개 모델 전체의 합집합(51/100)이 최선의 단일 모델보다 여전히 10.9% 높다는 점은, 확률적 다양성 원리가 최신 모델에서도 여전히 유효함을 확인시켜 준다.

6.5 한계 및 향후 연구

본 연구는 몇 가지 한계를 가진다. 첫째, 오픈소스 모델(Qwen 2.5 Coder 32B)의 평가가 파이프라인 초기 버전의 버그로 인해 공정하게 수행되지 못하였다. 둘째, 모든 실험이 단일 GPU(RTX PRO 6000 Blackwell) 환경에서 수행되어, 상이한 GPU 아키텍처에서의 일반화 가능성을 확인하지 못하였다. 셋째, 확장 실험에서 Anthropic의 claude-opus-4-7을 포함하였으나, Gemini 등 다른 LLM 제품군과의 비교가 부재하며, 확장 실험은 Level 1에 한정되어 있다.

향후 연구 방향으로 다음을 제안한다.

1. **오픈소스 모델 재평가**: 수정된 파이프라인으로 Qwen, DeepSeek-Coder, CodeLlama 등의 오픈소스 모델을 공정하게 재평가한다. 특히 TritonRL [27]과 AutoTriton [17]에서 보인 RL 기반 미세 조정(fine-tuning) 접근과의 비교를 수행한다.
2. **값 불일치 해결 전략**: 에러 전이 가설에 기반하여, 값 불일치 에러에 특화된 피드백 메커니즘을 설계한다. 구체적으로 중간 텐서 값 비교, 수치 안정성 분석, 그리고 TritonForge [14]에서 제안된 프로파일링 기반 최적화를 통합할 수 있다.
3. **하이브리드 접근**: LLM 기반 초기 코드 생성과 Mirage [28]와 같은 수퍼옵티마이저(superoptimizer) 기반 후처리를 결합하는 하이브리드 파이프라인을 탐구한다.
4. **RAG 기반 패턴 학습**: Retrieval-Augmented Generation을 통해 기존의 성공적인 Triton 커널 패턴을 LLM의 문맥(context)에 주입하여, 특히 Level 3 수준의 복잡한 커널 생성 능력을 향상시킨다.

7 결론

본 논문은 LLM 기반 Triton GPU 커널 자동 생성에 대한 체계적 평가를 통해, 단순한 성능 비교를 넘어 커널 코드 자동 생성의 근본적 특성에 관한 핵심 원리를 발견하였다.

첫째, **에러 전이 가설**을 제안하고 실증적으로 검증하였다. LLM 능력이 향상됨에 따라 컴파일 에러에서 값 불일치로 지배적 에러 유형이 전이되며, 이는 Triton DSL 문법 학습과 GPU 연산 의미론 이해 사이에 본질적 난이도 격차가 존재함을 시사한다. 확장 실험에서 claude-opus-4-7의 컴파일 에러율 0%와 값 불일치 65%는 이 가설을 더욱 강하게 뒷받침한다.

둘째, **확률적 다양성 원리**를 발견하였다. 반복의 깊이보다 독립적 생성의 다양성이 더 효과적이며, 5개 모델 합집합(51/100, 51%)이 최선의 단일 모델(46/100)보다 10.9% 높은 결과는 최신 세대 모델에서도 이 원리가 유효함을 확인시켜 준다.

셋째, **플래그십 수렴 현상(Flagship Convergence)**을 발견하였다. gpt-5.5와 claude-opus-4-7이 처음 44문제에서 정확히 동일한 37개 문제를 해결하였으며, claude-opus-4-7은 후반부 회복 능력을 통해 단일 모델 최고 성능(**46/100, 46%**)을 달성하였다. 이는 이전 3-모델 캐스케이딩 전략(43/100)을 단일 모델로 초과한 최초의 결과이다.

넷째, 추론 특화 모델(gpt-5.2-pro)이 코드 생성 과제에서 비효율적임을 정량적으로 입증하였다. 한편, Level 1에서 49%의 문제가 여전히 미해결이며, 특히 값 불일치가 지배적 병목이라는 발견은 GPU 연산의 의미적 이해 향상이 차세대 핵심 과제임을 명확히 한다.

본 연구의 프레임워크와 발견은 LLM 기반 GPU 커널 생성 연구 커뮤니티에 체계적인 평가 방법론과 향후 연구 방향을 제시한다.

Acknowledgments

본 연구는 정보통신기획평가원(IITP)의 지원을 받아 수행되었습니다.

References

[1] Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burber, et al. 2024. PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation. In *ASPLOS*.

[2] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. 2021. Program Synthesis with Large Language Models. *arXiv preprint arXiv:2108.07732* (2021).

[3] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating Large Language Models Trained on Code. In *arXiv preprint arXiv:2107.03374*.

[4] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Q. Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. TVM: An Automated End-to-End Optimizing Compiler for Deep Learning. In *USENIX Symposium on Operating Systems Design and Implementation*. https://api.semanticscholar.org/CorpusID:52939079

[5] Wentao Chen, Jiace Zhu, Qi Fan, Yehan Ma, and An Zou. 2025. CUDA-LLM: LLMs Can Write Efficient CUDA Kernels. *arXiv preprint arXiv:2506.09092* (2025). arXiv:2506.09092

[6] Tri Dao. 2023. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. *arXiv preprint arXiv:2307.08691* (2023). arXiv:2307.08691

[7] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *NeurIPS* (2022).

[8] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y. Wu, Y.K. Li, et al. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming – The Rise of Code Intelligence. *arXiv preprint arXiv:2401.14196* (2024).

[9] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Kai Dang, et al. 2024. Qwen2.5-Coder Technical Report. *arXiv preprint arXiv:2409.12186* (2024).

[10] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *ICLR* (2024).

[11] Sathvik Joel, Jie JW Wu, and Fatemeh H. Fard. 2024. A Survey on LLM-based Code Generation for Low-Resource and Domain-Specific Programming Languages. *arXiv preprint arXiv:2410.03981* (2024). arXiv:2410.03981

[12] David B. Kirk and Wen-mei W. Hwu. 2016. *Programming Massively Parallel Processors: A Hands-on Approach* (3rd ed.). Morgan Kaufmann.

[13] Robert Tjarko Lange, Aaditya Prasad, Qi Sun, Maxence Faldor, Yujin Tang, and David Ha. 2025. *The AI CUDA Engineer: Agentic CUDA Kernel Discovery, Optimization and Composition.* Technical Report. Sakana AI. https://pub.sakana.ai/ai-cuda-engineer/paper/

[14] Haonan Li, Keyu Man, Partha Kanuparthy, Hanning Chen, Wei Sun, Sreen Tallam, Chenguang Zhu, Kevin Zhu, and Zhiyun Qian. 2025. TritonForge: Profiling-Guided Framework for Automated Triton Kernel Optimization. *arXiv preprint arXiv:2512.09196* (2025). arXiv:2512.09196

[15] Jianling Li, Shangzhan Li, Zhenye Gao, Qi Shi, Yuxuan Li, Zefan Wang, Jiacheng Huang, Haojie Wang, Jianrong Wang, Xu Han, Zhiyuan Liu, and Maosong Sun. 2025. TritonBench: Benchmarking Large Language Model Capabilities for Generating Triton Operators. In *Findings of the Association for Computational Linguistics (ACL)*. arXiv:2502.14752

[16] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. 2023. StarCoder: may the source be with you! *Transactions on Machine Learning Research* (2023).

[17] Shangzhan Li, Zefan Wang, Ye He, Yuxuan Li, Qi Shi, Jianling Li, Yonggang Hu, Wanxiang Che, Xu Han, Zhiyuan Liu, and Maosong Sun. 2025. AutoTriton: Automatic Triton Programming with Reinforcement Learning in LLMs. *arXiv preprint arXiv:2507.05687* (2025). arXiv:2507.05687

[18] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. 2023. CodeGen: An Open Large Language Model for Code with Multi-Turn Program Synthesis. *ICLR* (2023).

[19] OpenAI. 2023. GPT-4 Technical Report. *arXiv preprint arXiv:2303.08774* (2023).

[20] OpenAI. 2024. Learning to Reason with LLMs. https://openai.com/index/learning-to-reason-with-llms/.

[21] Anne Ouyang, Simon Zheng, Jiahui Xin, Charles Jiang, Abraham Luo, Tianjun Stojanov, Zicheng Bai, Kunle Chen, Trevor Darrell, Joseph E. Gonzalez, and Ion Stoica. 2024. KernelBench: Can LLMs Write GPU Kernels?. In *NeurIPS 2024 Workshop on Foundation Model Interventions*.

[22] Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand, and Saman Amarasinghe. 2013. Halide: a language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. *SIGPLAN Not.* 48, 6 (jun 2013), 519–530. doi:10.1145/2499370.2462176

[23] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, et al. 2023. Code Llama: Open Foundation Models for Code. *arXiv preprint arXiv:2308.12950* (2023).

[24] Benjamin F. Spector, Simran Arora, Aaryan Singhal, Daniel Y. Fu, and Christopher Ré. 2024. ThunderKittens: Simple, Fast, and Adorable AI Kernels. *arXiv preprint arXiv:2410.20399* (2024). arXiv:2410.20399

[25] Shailja Thakur, Baleegh Ahmad, Zhenxing Fan, Hammond Pearce, Benjamin Tan, Ramesh Karri, Brendan Dolan-Gavitt, and Siddharth Garg. 2024. VeriGen: A Large Language Model for Verilog Code Generation. *ACM Transactions on Design Automation of Electronic Systems* 29, 3 (2024). doi:10.1145/3643681

[26] Philippe Tillet, H. T. Kung, and David Cox. 2019. Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations. *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages* (2019), 10–19.

[27] Jiin Woo, Shaowei Zhu, Allen Nie, Zhen Jia, Yida Wang, and Youngsuk Park. 2025. TritonRL: Training LLMs to Think and Code Triton Without Cheating. *arXiv preprint arXiv:2510.17891* (2025). arXiv:2510.17891

[28] Mengdi Wu, Xinhao Cheng, Shengyu Liu, Chunan Shi, Jianan Ji, Man Kit Ao, Praveen Velliengiri, Xupeng Miao, Oded Padon, and Zhihao Jia. 2025. Mirage: A Multi-Level Superoptimizer for Tensor Programs. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. arXiv:2405.05751

[29] Yang Yu, Peiyu Zang, Chi Hsu Tsai, Haiming Wu, Yixin Shen, Jialing Zhang, Haoyu Wang, Zhiyou Xiao, Jingze Shi, Yuyu Luo, Wentao Zhang, Chunlei Men, Guang Liu, and Yonghua Lin. 2026. Towards Automated Kernel Generation in the Era of LLMs. *arXiv preprint arXiv:2601.15727* (2026). arXiv:2601.15727

[30] Lianmin Zheng, Chengfan Jia, Minmin Sun, Zhao Wu, Cody Hao Yu, Ameer Haj-Ali, Yida Wang, Jun Yang, Danyang Zhuo, Koushik Sen, et al. 2020. Ansor: Generating high-performance tensor programs for deep learning. In *14th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 20)*. 863–879.